

Aplicación Web con Sistema de Gestión de Pruebas

Elkin Stiven Contreras Rojas

David Felipe Perdomo Castillo

David Santiago Gomez Gutierrez

Calidad de Software

Jose Miguel Llanos Mosquera

Programa de Ingeniería de Sistemas

Corporación Universitaria del Huila

19 de mayo de 2026

Tabla de Contenido

Introducción	2
Objetivos	5
Objetivo General	5
Objetivos Específicos	5
Desarrollo	6
Marco Conceptual: Calidad de Software	6
Descripción de la Aplicación Construida	7
Pruebas Unitarias con JUnit	9
Pruebas Funcionales con Selenium WebDriver	10
Pruebas de Rendimiento con JMeter	12
Análisis Estático con SonarQube	15
Integración y Automatización con Docker y Jenkins	16
Resultados Obtenidos	18
Resultados de Pruebas Unitarias y Cobertura	18
Resultados de Pruebas Funcionales	20
Resultados de Pruebas de Rendimiento	22
Resultados del Análisis Estático	24
Conclusiones	26
Referencias	28

Introducción

La calidad del software se ha convertido en un factor determinante para el éxito de cualquier producto tecnológico moderno. A medida que las aplicaciones web crecen en tamaño y complejidad, también lo hace la necesidad de garantizar que funcionen correctamente, que sean seguras y que ofrezcan un buen rendimiento incluso cuando muchas personas las usan al mismo tiempo. Por esta razón, los ingenieros de software ya no solo desarrollan aplicaciones, sino que

también diseñan procesos completos para verificar y validar que el producto cumpla con lo que el usuario espera de él.

El presente trabajo describe el desarrollo de un proyecto integrador realizado en la asignatura Calidad de Software, en el cual se construyó una aplicación web tipo gestor de tareas y proyectos, acompañada de un sistema completo de pruebas automatizadas. La aplicación fue desarrollada con el lenguaje Java y el framework Spring Boot, y su interfaz de usuario se elaboró con HTML, CSS y JavaScript nativos. El objetivo no era únicamente construir una aplicación funcional, sino aplicar de manera práctica los estándares y herramientas que la industria utiliza para asegurar la calidad del software.

Para lograrlo, se implementaron cuatro tipos de pruebas distintas, cada una enfocada en un aspecto diferente del producto. Las pruebas unitarias, escritas con JUnit, verifican que cada pequeña parte del código funcione correctamente de manera aislada. Las pruebas funcionales, realizadas con Selenium WebDriver, simulan a un usuario real que abre el navegador, hace clic en los botones y completa formularios. Las pruebas de rendimiento, ejecutadas con JMeter, miden cómo se comporta la aplicación cuando muchos usuarios la utilizan al mismo tiempo. Finalmente, el análisis estático con SonarQube revisa el código fuente buscando errores, malas prácticas y posibles vulnerabilidades de seguridad sin necesidad de ejecutarlo.

Todo este conjunto de pruebas se integró en un pipeline de integración continua usando Docker y Jenkins. Esto significa que cada vez que se realiza un cambio en el código, automáticamente se compila la aplicación, se ejecutan todas las pruebas y se generan reportes con los resultados. Esta automatización es esencial en el desarrollo moderno porque permite detectar errores temprano y reducir el tiempo que se invierte en pruebas manuales.

El trabajo también se apoya en estándares internacionales reconocidos por la comunidad del software, como la norma ISO/IEC 25010, que define las características que debe tener un producto de calidad, y la norma ISO/IEC 29119, que establece cómo deben planificarse y documentarse los procesos de prueba. Aplicar estos estándares permite que el proyecto no se limite

a una experiencia académica, sino que se aproxime a la forma en que se trabaja en entornos profesionales reales.

En las páginas siguientes se presenta primero un marco conceptual con las ideas fundamentales sobre calidad de software, seguido por una descripción detallada de cómo se construyó cada una de las pruebas y, finalmente, los resultados obtenidos al ejecutar el pipeline completo. El documento cierra con un conjunto de conclusiones que recogen las lecciones aprendidas durante el proceso y reflexiona sobre los aspectos que aún pueden mejorarse en futuras iteraciones del proyecto.

Objetivos

Objetivo General

Aplicar de manera integral los conceptos, estándares internacionales y herramientas modernas de aseguramiento y control de calidad de software en el desarrollo y validación de una aplicación web construida con Java y Spring Boot, demostrando competencia técnica y conceptual en cada una de las etapas del proceso.

Objetivos Específicos

Desarrollar una aplicación web funcional de gestión de tareas y proyectos que incluya autenticación de usuarios, operaciones de creación, lectura, actualización y eliminación sobre las entidades principales, así como una interfaz gráfica accesible desde el navegador.

Implementar una suite de pruebas unitarias con JUnit 5 que verifique el comportamiento de la lógica de negocio del sistema, alcanzando una cobertura significativa en la capa de servicios y documentando los casos exitosos, los casos límite y los escenarios de error esperados.

Diseñar y ejecutar pruebas funcionales automatizadas con Selenium WebDriver siguiendo el patrón Page Object Model, cubriendo los flujos críticos de la aplicación tanto en Chrome como en Firefox, para validar la experiencia real del usuario final.

Realizar pruebas de rendimiento y carga con Apache JMeter sobre los endpoints más utilizados del sistema, midiendo tiempos de respuesta, rendimiento y tasa de error bajo diferentes niveles de concurrencia, con el fin de identificar cuellos de botella y puntos de mejora.

Integrar SonarQube como herramienta de análisis estático para revisar la calidad del código fuente, detectar duplicaciones, malas prácticas y posibles vulnerabilidades, y documentar los hallazgos junto con un plan de mejora orientado a reducir la deuda técnica del proyecto.

Configurar un pipeline de integración continua en Jenkins que orqueste, dentro de contenedores Docker, la compilación, ejecución de pruebas y generación de reportes, garantizando que el proceso sea reproducible y se ejecute automáticamente con cada cambio en el código.

Desarrollo

Marco Conceptual: Calidad de Software

La calidad de software puede entenderse como el grado en el cual un producto cumple con los requisitos establecidos por sus usuarios y con las expectativas implícitas de quienes lo utilizan. No se trata únicamente de que la aplicación funcione, sino de que lo haga bien: que sea confiable, que se pueda mantener, que sea segura y que ofrezca una buena experiencia. Pressman y Maxim (2020) destacan que la calidad no es un atributo que se añade al final del desarrollo, sino una propiedad que debe construirse desde el inicio y verificarse de manera continua a lo largo de todo el ciclo de vida del software.

Dentro de este campo es común distinguir entre dos conceptos relacionados pero diferentes: el aseguramiento de la calidad o QA, por sus siglas en inglés, y el control de calidad o QC. El aseguramiento se enfoca en los procesos: revisa cómo se está construyendo el software, qué estándares se siguen y qué prácticas se aplican para prevenir defectos. El control, por su parte, se enfoca en el producto: verifica el resultado final mediante pruebas y mediciones para detectar errores antes de que lleguen al usuario. Ambos enfoques son complementarios y, en proyectos modernos, trabajan de la mano dentro de pipelines automatizados.

Uno de los marcos más utilizados para hablar de calidad de software es la norma ISO/IEC 25010, que reemplazó al modelo anterior ISO/IEC 9126. Esta norma identifica ocho características principales que debe poseer un producto de calidad: adecuación funcional, eficiencia de desempeño, compatibilidad, usabilidad, fiabilidad, seguridad, mantenibilidad y portabilidad (ISO/IEC, 2011). Cada una de estas características se subdivide en subcaracterísticas más específicas que permiten medir aspectos concretos del software. En el presente proyecto se han trabajado especialmente la adecuación funcional mediante las pruebas unitarias y funcionales, la eficiencia de desempeño mediante las pruebas de carga con JMeter, la mantenibilidad mediante el análisis estático con SonarQube, y la fiabilidad mediante la combinación de todas las pruebas anteriores.

Otra norma de gran importancia en este contexto es la ISO/IEC 29119, que establece un marco internacional para los procesos de prueba de software. Esta norma propone una estructura ordenada para planificar, diseñar, ejecutar y reportar las pruebas, e introduce conceptos como el plan maestro de pruebas, la especificación de casos y la trazabilidad entre requerimientos y pruebas. Si bien en un proyecto académico no se implementa esta norma en su totalidad, sus principios sirven como guía para organizar el trabajo de manera profesional.

Las métricas también juegan un papel central en la calidad del software. Entre las métricas más utilizadas se encuentran la cobertura de código, que indica qué porcentaje del código fuente fue ejecutado por las pruebas; la densidad de defectos, que mide cuántos errores se encuentran por unidad de código; el tiempo medio de recuperación (MTTR), que estima cuánto tarda el sistema en restablecerse tras un fallo; y la deuda técnica, que cuantifica el costo estimado de corregir todas las malas prácticas detectadas. Estas métricas no son fines en sí mismas, pero permiten tomar decisiones informadas sobre dónde invertir el esfuerzo de mejora.

Finalmente, conviene mencionar las herramientas que hacen posible aplicar todos estos conceptos en la práctica. JUnit es el framework más utilizado para escribir pruebas unitarias en Java; Selenium WebDriver permite automatizar interacciones con navegadores; JMeter es una herramienta abierta para pruebas de rendimiento; SonarQube ofrece análisis estático del código; Docker facilita ejecutar todo el entorno de manera reproducible en contenedores; y Jenkins permite orquestar el flujo completo en un pipeline automático. La combinación de estas herramientas es la base del trabajo realizado en este proyecto.

Descripción de la Aplicación Construida

La aplicación desarrollada se denomina Task Manager y consiste en un sistema web para gestionar proyectos colaborativos y las tareas asociadas a cada uno de ellos. La idea general se inspira en herramientas reales del mercado, como Trello o Asana, aunque simplificada para los fines del proyecto académico. Cada usuario puede registrarse, iniciar sesión, crear sus propios proyectos, invitar a otros usuarios como miembros, crear tareas dentro de cada proyecto y moverlas a través de un tablero con tres estados: Pendiente, En progreso y Terminado.

Desde el punto de vista técnico, la aplicación se construyó con Spring Boot 4.0.6 ejecutándose sobre Java 21. La capa de persistencia utiliza Spring Data JPA contra una base de datos PostgreSQL 15. La autenticación se resuelve mediante tokens JWT firmados con la librería JJWT, con una expiración configurada en veinticuatro horas. La validación de los datos de entrada se realiza con Spring Validation y las anotaciones estándar de Bean Validation. Para reducir el código repetitivo en las entidades y los DTOs se utilizó Lombok, una librería que genera automáticamente métodos como los getters, los setters y los constructores.

El frontend se construyó dentro de la carpeta `resources/static` del propio proyecto Spring Boot, utilizando HTML, CSS y JavaScript nativos sin frameworks adicionales como React o Angular. Esta decisión, aunque menos común en aplicaciones grandes, resulta adecuada para un proyecto académico porque permite mantener todo en un único repositorio y reduce la complejidad del despliegue. Las páginas del frontend consumen los endpoints REST expuestos por el backend y manejan la autenticación almacenando el token JWT en el navegador.

El modelo de datos se compone de cuatro entidades principales. La entidad `User` representa a un usuario del sistema con su nombre de usuario único, su correo electrónico, su contraseña hashada con BCrypt y la fecha de creación de la cuenta. La entidad `Project` representa un proyecto, que pertenece a un usuario propietario o *owner*. La entidad `Task` representa una tarea dentro de un proyecto y guarda información como el título, la descripción, el estado actual, quién la creó y a quién está asignada. Finalmente, la entidad `ProjectMember` modela la relación de membresía entre usuarios y proyectos, permitiendo que varios usuarios trabajen colaborativamente sobre un mismo proyecto.

Una decisión de diseño importante fue manejar los roles de acceso a nivel del proyecto y no a nivel global del usuario. Esto significa que no existen roles como “administrador” o “usuario común” en la base de datos: en su lugar, cada usuario es *owner* de los proyectos que crea y miembro de los proyectos a los que ha sido invitado. Las reglas de negocio del sistema verifican estos vínculos antes de permitir cada operación. Por ejemplo, solo el *owner* de un proyecto puede

eliminarlo o modificar sus datos, mientras que cualquier miembro puede crear y editar tareas dentro de ese proyecto.

Toda la infraestructura del proyecto se ejecuta dentro de Docker Compose, un orquestador que permite levantar varios contenedores con un único comando. Los servicios incluidos son la aplicación principal expuesta en el puerto 8081, la base de datos PostgreSQL, una instancia de SonarQube en el puerto 9000 y un servidor Jenkins en el puerto 8090. Adicionalmente, para las pruebas funcionales se levanta de manera complementaria un Selenium Grid con los navegadores Chrome y Firefox. Esta arquitectura en contenedores hace que el proyecto pueda ejecutarse en cualquier máquina con Docker instalado, sin tener que configurar manualmente cada herramienta por separado.

Pruebas Unitarias con JUnit

Las pruebas unitarias son la primera y más numerosa línea de defensa contra los errores en un proyecto de software. Su propósito es verificar que cada unidad pequeña del código, generalmente un método dentro de una clase, se comporte exactamente como se espera frente a una entrada dada. Lo característico de este tipo de pruebas es que deben ser rápidas, independientes y deterministas, lo que significa que no deberían depender de bases de datos reales, ni de servicios externos, ni del orden en que se ejecutan.

En este proyecto se utilizó JUnit 5 junto con Mockito, una librería que permite crear objetos simulados llamados *mocks*. Gracias a Mockito se pueden probar los servicios sin necesidad de conectarse a la base de datos real: en lugar de eso, se le indica al *mock* qué debe devolver cuando se invoca cierto método, y luego se verifica que el servicio reaccione correctamente ante esa respuesta simulada. Este enfoque es el estándar de la industria para pruebas unitarias en aplicaciones Spring Boot.

Las pruebas se organizaron en cinco suites principales que reflejan la estructura del código de producción. La suite `UserServiceTest` cubre el registro de usuarios, el inicio de sesión, la generación de tokens JWT y la obtención del perfil. La suite `ProjectServiceTest` agrupa todas las operaciones relacionadas con proyectos, incluyendo la creación, edición, eliminación, invitación

de miembros, remoción de miembros y consulta de detalles. La suite `TaskServiceTest` cubre las operaciones sobre tareas: creación, edición, eliminación, asignación a usuarios y transición entre los tres estados del tablero. La suite `JwtServiceTest` se centra exclusivamente en la generación y validación de tokens, incluyendo casos de tokens expirados y con firma alterada. Finalmente, una suite de contexto de Spring asegura que la aplicación pueda arrancar correctamente.

Cada caso de prueba sigue un patrón conocido como *Given-When-Then*, traducido al español como “Dado, Cuando, Entonces”. En la sección *Dado* se preparan los datos y los *mocks* necesarios; en la sección *Cuando* se invoca el método que se quiere probar; y en la sección *Entonces* se verifican los resultados con aserciones. Este patrón hace que las pruebas sean fáciles de leer y entender, incluso para alguien que se está iniciando en el tema. Para facilitar la comprensión, todos los comentarios dentro de las pruebas se redactaron en español.

Un detalle importante del trabajo fue distinguir entre validaciones que pertenecen a la capa de servicio y validaciones que pertenecen a la capa de controlador. Las anotaciones como `@NotBlank`, `@Size` y `@Email`, que validan los datos de entrada, son ejecutadas por Spring antes de que la petición llegue al servicio. Por lo tanto, no tiene sentido probarlas con pruebas unitarias del servicio, ya que en ese momento los datos ya pasaron la validación. Estas pruebas corresponden a un tipo diferente, llamado pruebas de controlador o de integración web, que quedaron identificadas como mejora para futuras iteraciones del proyecto.

Para medir qué porcentaje del código fue ejecutado por las pruebas se utilizó JaCoCo, una herramienta que se integra con Maven y genera un reporte HTML detallado. Este reporte muestra, paquete por paquete y clase por clase, qué líneas y qué ramas condicionales fueron cubiertas por las pruebas. La cobertura se interpretará en la sección de resultados, pero conviene anticipar que la capa de servicios alcanzó valores cercanos al noventa y cuatro por ciento, lo que es considerado excelente según los estándares de la industria.

Pruebas Funcionales con Selenium WebDriver

Las pruebas funcionales o de interfaz se ubican en un nivel más alto que las pruebas unitarias. Mientras estas últimas verifican piezas pequeñas del código, las pruebas funcionales

verifican flujos completos de usuario, simulando lo que haría una persona real al usar la aplicación. Para lograrlo se utilizó Selenium WebDriver, una herramienta que permite controlar navegadores web mediante código y que es ampliamente utilizada en la industria para automatizar este tipo de pruebas.

En el proyecto se diseñaron cinco escenarios funcionales que cubren los flujos más críticos del sistema. El primero, llamado AuthFlowTest, valida el ciclo completo de autenticación: registro de un nuevo usuario, inicio de sesión y cierre de sesión. El segundo, ProfileViewTest, verifica que el usuario pueda visualizar su propio perfil. El tercero, ProjectCrudTest, comprueba que un usuario pueda crear, editar y eliminar un proyecto. El cuarto, MembershipTest, valida el flujo de invitar a otro usuario, asignarle una tarea y finalmente removerlo del proyecto. El quinto, TaskKanbanTest, verifica que una tarea creada pueda moverse por los tres estados del tablero Kanban.

Cada uno de estos cinco escenarios se ejecuta dos veces, una en Chrome y otra en Firefox, lo que da un total de diez ejecuciones por corrida del pipeline. La razón para ejecutar las pruebas en dos navegadores diferentes es asegurar que la aplicación funcione correctamente en ambos motores de renderizado, ya que existen diferencias sutiles entre Chromium y Gecko que pueden afectar comportamientos específicos del frontend.

Para mantener el código de las pruebas organizado y fácil de mantener se aplicó el patrón Page Object Model, conocido como POM por sus siglas en inglés. Este patrón consiste en crear una clase por cada página o pantalla de la aplicación, encapsulando en ella los selectores HTML y los métodos de interacción. De esta manera, si en el futuro cambia un botón o un campo del formulario, basta con actualizar el archivo de esa página y todas las pruebas que la usan seguirán funcionando sin modificación. Este patrón es considerado una buena práctica esencial en proyectos de automatización a mediana o gran escala.

La ejecución de Selenium dentro del pipeline plantea un desafío técnico: necesita navegadores instalados en el entorno donde corre, y los contenedores típicos de Jenkins no los incluyen. La solución adoptada fue utilizar Selenium Grid, una arquitectura distribuida que permite

levantar contenedores especializados con los navegadores ya instalados. En este proyecto se levantaron dos nodos, uno para Chrome y otro para Firefox, conectados a un hub central. Las pruebas no se ejecutan directamente en el contenedor del job, sino que envían sus comandos al hub, que los redirige al nodo correspondiente. Toda esta red ocurre dentro de Docker, lo que la hace reproducible y aislada del entorno del desarrollador.

Durante el desarrollo de las pruebas se enfrentaron varios obstáculos técnicos típicos de Selenium. Uno de ellos fue el manejo de mensajes emergentes que aparecen después de ciertas acciones, como el mensaje de confirmación tras registrar una cuenta, que bloqueaba al robot. La solución consistió en programar la prueba para cerrar el mensaje antes de continuar. Otro problema fue la velocidad del navegador Chrome, que en algunas pruebas leía el DOM mientras este se estaba actualizando, generando lecturas inconsistentes; esto se solucionó implementando esperas explícitas con `WebDriverWait`, que reintentan la consulta hasta que el elemento alcance el estado esperado o se cumpla un tiempo límite.

Otro detalle de calidad fue la generación automática de capturas de pantalla cuando alguna prueba falla. Estas imágenes se guardan en una carpeta del proyecto y permiten diagnosticar visualmente qué ocurrió en el momento del error, sin tener que reproducir manualmente toda la prueba. En la corrida final del pipeline ninguna prueba falló, por lo que la carpeta de capturas quedó vacía, pero el mecanismo está disponible para futuras ejecuciones.

Pruebas de Rendimiento con JMeter

Las pruebas de rendimiento responden a una pregunta distinta a la de las pruebas funcionales. No se trata ya de verificar si la aplicación hace lo correcto, sino de medir qué tan bien lo hace bajo distintas condiciones de carga. En particular, interesa saber qué pasa cuando muchos usuarios usan el sistema al mismo tiempo: si los tiempos de respuesta se mantienen aceptables, si aparecen errores y, si los hay, en qué porcentaje. Para realizar este tipo de mediciones se utilizó Apache JMeter, una herramienta libre y madura que permite simular cargas de cientos o miles de usuarios virtuales contra un servicio web.

Se diseñó un plan de pruebas que sigue una estructura de tres escenarios complementarios, con un total aproximado de noventa y cinco usuarios virtuales escalando en oleadas a lo largo de poco más de dos minutos. El primer escenario corresponde a una carga normal de cinco usuarios concurrentes que realizan operaciones cotidianas: iniciar sesión, consultar su perfil, listar sus proyectos y obtener las tareas de un proyecto pequeño. Este escenario establece una línea base de comportamiento, contra la cual se comparan los otros.

El segundo escenario simula una situación de carga elevada con treinta usuarios concurrentes que repetidamente consultan las tareas de un proyecto grande, con cien tareas. Este escenario fue diseñado deliberadamente para exponer un problema clásico en aplicaciones JPA conocido como problema N+1: cuando una entidad tiene relaciones cargadas de manera perezosa o *lazy* y la aplicación accede a cada una de ellas individualmente, en lugar de hacerlo con una sola consulta optimizada. El resultado es que una petición que aparenta ser simple genera internamente muchas consultas a la base de datos.

El tercer escenario representa una prueba de estrés con sesenta usuarios concurrentes y una rampa de subida muy rápida de cinco segundos. Esta configuración fuerza al sistema más allá de su capacidad nominal y permite observar cómo se degrada bajo presión extrema. En particular, se esperaba ver fallos asociados a la saturación del pool de conexiones de HikariCP, el componente de Spring Boot encargado de administrar las conexiones a la base de datos, así como dificultades en el hashing BCrypt de las contraseñas durante los logins concurrentes.

Para distinguir los resultados de cada escenario en el reporte final, cada solicitud fue etiquetada con un prefijo entre corchetes: NORMAL, CARGA o ESTRES. De esta manera, al revisar los gráficos generados por JMeter es posible filtrar por etiqueta y analizar cada caso por separado. Los resultados detallados se presentan en la sección correspondiente, pero conviene adelantar que el plan cumplió su propósito: los dos cuellos de botella esperados aparecieron con claridad y permitieron formular recomendaciones concretas de optimización.

Adicionalmente, en una de las corridas integradas dentro del pipeline de Jenkins se ejecutó un plan unificado con noventa y cinco hilos en oleadas continuas, sin separar los tres escenarios.

Esta ejecución generó un volumen de más de veintidós mil peticiones en aproximadamente ciento treinta y cinco segundos. Los datos obtenidos permitieron observar la evolución del rendimiento a medida que crecía la concurrencia, y resultaron consistentes con lo hallado en los tres escenarios separados: la aplicación responde adecuadamente hasta unos treinta usuarios concurrentes y comienza a presentar errores significativos a partir de los sesenta.

Análisis Estático con SonarQube

El análisis estático se diferencia de las pruebas anteriores en un aspecto clave: no ejecuta el código. En lugar de eso, lo examina como si fuera un texto, buscando patrones que sugieran posibles problemas, malas prácticas o vulnerabilidades. Esta técnica es complementaria a las pruebas dinámicas y permite detectar problemas que podrían pasar desapercibidos durante la ejecución, especialmente aquellos relacionados con la calidad del código fuente y la mantenibilidad a largo plazo.

En este proyecto se utilizó SonarQube en su versión comunitaria, ejecutándose en un contenedor Docker independiente. SonarQube clasifica los problemas detectados en tres categorías principales: bugs, que son errores funcionales reales; vulnerabilidades, que son problemas de seguridad explotables; y code smells, que son malas prácticas o decisiones de diseño que no rompen el código pero dificultan su mantenimiento. Adicionalmente, marca como *security hotspots* ciertos fragmentos sospechosos que requieren revisión manual antes de confirmarse como vulnerabilidades.

La herramienta también asigna un rating de la A a la E a cada una de estas categorías, donde A representa el mejor estado y E el peor. Estos ratings se calculan en función del número y severidad de los problemas encontrados, y son útiles para tener una visión rápida del estado del proyecto. Adicionalmente, SonarQube calcula una métrica llamada deuda técnica, expresada en unidades de tiempo, que estima cuánto tomaría resolver todos los problemas detectados.

Un concepto importante es el de *Quality Gate* o puerta de calidad. Se trata de un conjunto de condiciones que el código debe cumplir para considerarse aceptable. La configuración por defecto exige, por ejemplo, que no haya bugs, que la cobertura sea superior a un umbral mínimo y que la duplicación de código no exceda cierto porcentaje. Si la *Quality Gate* falla, el pipeline marca el build como rechazado. En este proyecto la puerta de calidad fue aprobada en la corrida final, lo que indica que el código cumple con los criterios mínimos establecidos.

Integración y Automatización con Docker y Jenkins

Toda la complejidad descrita en las secciones anteriores no tendría mucho valor si cada prueba debiera ejecutarse manualmente. El verdadero potencial de un sistema de calidad moderno aparece cuando todo se integra en un pipeline automatizado, capaz de ejecutarse con un solo clic o de manera completamente desatendida cuando se sube un cambio al repositorio. Para lograr este nivel de automatización se combinaron dos tecnologías: Docker para contenerizar cada componente, y Jenkins para orquestar la secuencia completa de pasos.

Docker permite empaquetar una aplicación junto con todas sus dependencias en un contenedor que se ejecuta de manera idéntica en cualquier sistema operativo. Esto elimina el clásico problema del “funciona en mi máquina”, ya que el contenedor se comporta igual en el computador del desarrollador, en el servidor de pruebas y en producción. En este proyecto se utilizó Docker Compose para definir todos los servicios necesarios en un único archivo de configuración: la aplicación Spring Boot, la base de datos PostgreSQL, SonarQube, Jenkins y, durante la ejecución de pruebas funcionales, los nodos de Selenium Grid.

Jenkins es uno de los servidores de integración continua más utilizados en la industria, conocido por su flexibilidad y por su amplio ecosistema de plugins. En este proyecto se configuró un pipeline declarativo escrito en un archivo *Jenkinsfile* que define cada etapa del proceso. Las etapas incluyen la descarga del código desde GitHub, la compilación con Maven, la ejecución de pruebas unitarias, la generación del reporte de cobertura con JaCoCo, el análisis estático con SonarQube, las pruebas funcionales con Selenium y las pruebas de rendimiento con JMeter.

Una decisión técnica relevante fue construir una imagen personalizada de Jenkins que incluyera Java 21 instalado, ya que la imagen oficial no traía la versión adecuada para compilar el proyecto. Esto se logró creando un *Dockerfile* propio que parte de la imagen base de Jenkins y agrega el JDK, junto con otras utilidades necesarias. Este pequeño detalle ilustra cómo, en proyectos reales, la integración de herramientas frecuentemente requiere ajustes que no aparecen explícitamente en la documentación oficial.

Otro aspecto clave es la comunicación entre contenedores. Dentro de la red de Docker, cada contenedor puede ser localizado por su nombre, en lugar de por una dirección IP. Por ejemplo, Jenkins se refiere a la aplicación como *docker-app-1* y al hub de Selenium como *selenium-hub*. Esta nomenclatura simplifica enormemente la configuración, ya que se mantiene estable incluso cuando los contenedores se reinician con direcciones IP diferentes.

Al finalizar cada ejecución del pipeline, Jenkins archiva los artefactos producidos: el reporte de cobertura HTML, el reporte de SonarQube, los gráficos de JMeter, los reportes Surefire de las pruebas unitarias y los reportes de Selenium. Esto permite que cualquier miembro del equipo, no solo quien ejecutó el pipeline, pueda consultar los resultados a través de la interfaz web de Jenkins. En la corrida número trece del pipeline, todas las etapas finalizaron correctamente y el build se marcó como SUCCESS, lo que confirma que el sistema completo funciona como se esperaba.

Resultados Obtenidos

Resultados de Pruebas Unitarias y Cobertura

La suite completa de pruebas unitarias consta de sesenta y ocho casos distribuidos en cinco grupos según el componente que verifican. La ejecución total tomó alrededor de diez segundos y, en la corrida final del pipeline, las sesenta y ocho pruebas pasaron satisfactoriamente, sin fallos, errores ni casos omitidos. Esto representa una tasa de éxito del cien por ciento, lo cual constituye una primera señal positiva sobre la estabilidad de la lógica de negocio del sistema.

Tabla 1

Distribución de pruebas unitarias por suite y tiempo de ejecución

ID	Suite	Pruebas	Tiempo (s)
TU-01	UserService — Gestión de Usuarios	8	0.189
TU-02	ProjectService — Gestión de Proyectos	28	0.167
TU-03	TaskService — Gestión de Tareas	26	2.232
TU-05	JwtService — Seguridad y Tokens JWT	5	0.855
—	AppApplicationTests (contexto Spring)	1	7.346
Total		68	~10.8

Nota. Elaboración propia a partir del reporte Surefire de Jenkins build #13.

Respecto a la cobertura de código medida con JaCoCo, los resultados muestran una distribución desigual entre los distintos paquetes del proyecto, lo cual es esperado dada la estrategia adoptada de concentrar el esfuerzo en la capa de servicios. La cobertura global alcanzó el setenta y cuatro por ciento de instrucciones y el cincuenta y nueve por ciento de ramas condicionales, mientras que la capa de servicios, donde reside la lógica de negocio principal, alcanzó valores cercanos al noventa y cuatro por ciento, considerados excelentes según los estándares de la industria.

Los paquetes con menor cobertura corresponden a los controladores, con cero por ciento de cobertura unitaria, y al manejador global de excepciones, con apenas un veintidós por ciento. Estos valores no deben interpretarse como un problema crítico, ya que los controladores son validados indirectamente por las pruebas funcionales de Selenium, que recorren los endpoints reales del sistema. No obstante, sí representan una oportunidad clara de mejora mediante la incorporación de pruebas de integración web con MockMvc o WebTestClient, que permitirían cubrir esos paquetes con pruebas automatizadas más rápidas.

Resultados de Pruebas Funcionales

Las pruebas funcionales con Selenium se ejecutaron en dos navegadores diferentes, Chrome y Firefox, totalizando diez ejecuciones por corrida del pipeline. En la corrida final, las diez ejecuciones finalizaron satisfactoriamente, sin fallos ni errores. El tiempo total de ejecución fue de aproximadamente noventa y un segundos, que es un valor razonable considerando que cada escenario implica abrir el navegador, navegar por varias páginas y completar varios formularios.

Tabla 2

Resultados de las pruebas funcionales por escenario

ID Escenario	Descripción	Requerimientos cubiertos	Tiempo (s)
AuthFlowTest	Registro, login y logout	RF-01.1 a RF-01.3	10.77
ProfileViewTest	Visualización del perfil	RF-01.4	8.87
ProjectCrudTest	Crear, editar y eliminar proyecto	RF-02.1, 02.4, 02.5	14.70
MembershipTest	Invitar, asignar y remover miembro	RF-02.6 a 02.8, RF-03.5	37.68
TaskKanbanTest	Crear tarea y moverla por el Kanban	RF-03.1, 03.4	18.81
Total (10 ejecuciones)		—	~91

Nota. Elaboración propia a partir del reporte de Selenium Grid en Jenkins build #13. Cada escenario fue ejecutado dos veces, una en Chrome y otra en Firefox.

Una observación interesante es que el escenario MembershipTest fue el más extenso en tiempo, casi cuarenta segundos, lo cual es coherente con la cantidad de pasos que involucra: registrar dos usuarios diferentes, iniciar sesión con uno, invitar al otro, asignar tareas, y finalmente removerlo. Cada uno de estos pasos requiere esperas explícitas para asegurar que el frontend haya actualizado el estado antes de continuar, lo cual incrementa el tiempo total pero es necesario para la estabilidad de la prueba.

Vale la pena destacar que los controladores, que tenían cero por ciento de cobertura según las pruebas unitarias, fueron efectivamente ejercitados durante estas pruebas funcionales. Esto

significa que el cero por ciento mostrado en JaCoCo es un valor técnico que solo refleja las pruebas unitarias, no la cobertura real del sistema. Si se integraran ambos tipos de cobertura, la cifra global sería considerablemente más alta.

Resultados de Pruebas de Rendimiento

Las pruebas de rendimiento revelaron información valiosa sobre los límites del sistema. La aplicación se comporta de manera excelente con cargas moderadas y comienza a degradarse claramente cuando la concurrencia supera los treinta usuarios simultáneos. A continuación se presentan los resultados desglosados por escenario obtenidos en la ejecución del plan de pruebas con los tres escenarios diferenciados.

Tabla 3

Resultados de las pruebas de rendimiento por escenario

Escenario	GET tasks promedio	GET tasks p95	Login promedio	Tasa de error
Normal (5 usuarios)	6 ms	8 ms	72 ms	0%
Carga (30 usuarios)	137 ms	264 ms	178 ms	0%
Estrés (60 usuarios)	204 ms	540 ms	259 ms	36.5%

Nota. Elaboración propia con base en el reporte HTML generado por JMeter. La columna p95 indica el percentil 95 del tiempo de respuesta, es decir, el valor por debajo del cual se encuentra el 95% de las peticiones.

El primer hallazgo importante es la degradación de veintitrés veces en el tiempo de respuesta del endpoint GET de tareas al pasar del escenario normal al de carga, sin que se generen errores. Este comportamiento es la huella característica del problema N+1 mencionado anteriormente: las relaciones perezosas *createdBy* y *assignedTo* de la entidad Task disparan aproximadamente doscientas una consultas adicionales por cada petición a la base de datos. La solución propuesta es modificar el repositorio para usar consultas con JOIN FETCH, lo que reduciría todas esas consultas a una sola.

El segundo hallazgo, observado en el escenario de estrés, fue una tasa de error del treinta y seis con cinco por ciento, distribuida de manera equitativa entre errores 401 al iniciar sesión y errores 403 al consultar las tareas con un token inválido. Este patrón indica saturación combinada del pool de conexiones de HikariCP y del proceso de hashing BCrypt durante los logins

concurrentes. La recomendación es aumentar el tamaño máximo del pool desde el valor por defecto de diez hasta treinta conexiones, lo que debería absorber gran parte de la presión.

En la ejecución integrada al pipeline, con noventa y cinco hilos en oleadas continuas, se obtuvo un volumen total de veintidós mil cuatrocientas cincuenta y cinco peticiones con un rendimiento promedio de ciento sesenta y cinco peticiones por segundo. La tasa de error global fue del veintiuno con cinco por ciento, lo cual es consistente con los resultados separados: con cargas moderadas no hay errores, pero al sobrepasar los sesenta usuarios concurrentes la tasa se dispara rápidamente.

Resultados del Análisis Estático

El análisis estático con SonarQube se ejecutó sobre sesenta y cuatro archivos Java más un archivo XML de configuración. La *Quality Gate* resultó aprobada, lo cual indica que el código cumple con los criterios de calidad mínimos definidos. Las métricas globales fueron favorables, con ceros en bugs y vulnerabilidades, y una duplicación de código del cero por ciento, lo que es un excelente indicador de la calidad estructural del proyecto.

Tabla 4

Métricas globales de SonarQube para el proyecto

Métrica	Valor	Rating
Líneas de código (LOC)	1.4k	—
Issues de seguridad	0	A
Issues de fiabilidad	0	A
Issues de mantenibilidad	20	A
Cobertura	73.2%	—
Duplicación	0.0% (sobre 1.7k líneas)	—
Security Hotspots	1	E
Deuda técnica total	2h 17min	—

Nota. Tomado del dashboard de SonarQube tras el análisis ejecutado por el pipeline Jenkins build #13. El rating va de A (mejor) a E (peor).

Los veinte issues de mantenibilidad se distribuyen en cuatro niveles de severidad. Dos issues están clasificados como críticos: el primero corresponde al manejador global de excepciones, donde una cadena literal aparece duplicada cinco veces y debería extraerse como constante; el segundo corresponde a un método de prueba vacío sin justificación. Ambos pueden resolverse en menos de veinte minutos. Otros ocho issues están clasificados como mayores, e incluyen el uso de la excepción genérica *Exception* en lugar de excepciones específicas, bloques

de código vacíos en algunas Page Objects, y lambdas en las pruebas que invocan más de un método susceptible de lanzar excepción.

Los issues restantes son seis menores y cuatro informativos, entre los que aparecen usos de métodos deprecados de Selenium, paréntesis innecesarios y modificadores *public* redundantes en métodos de JUnit 5. Aunque ninguno de estos issues representa un problema funcional, su corrección mejoraría la deuda técnica acumulada, estimada en dos horas y diecisiete minutos, lo cual es un valor bajo para un proyecto de mil cuatrocientas líneas de código.

El *Security Hotspot* identificado, con rating E, no representa una vulnerabilidad confirmada sino un punto que SonarQube marca como sospechoso y requiere revisión manual. Tras inspeccionarlo en el dashboard se concluyó que corresponde a un uso aceptable en el contexto del proyecto, aunque queda como tarea pendiente formalizar esa revisión para que el hotspot pase a estado *safe*.

Conclusiones

El desarrollo de este proyecto integrador permitió consolidar de manera práctica los conceptos vistos a lo largo de la asignatura de Calidad de Software. Más allá de construir una aplicación web funcional, el verdadero aprendizaje estuvo en integrar distintas herramientas y enfoques de prueba dentro de un mismo flujo de trabajo automatizado, que es precisamente la forma en que se trabaja en entornos profesionales modernos. A continuación se presentan las principales lecciones obtenidas.

En primer lugar, el pipeline completo finalizó en estado SUCCESS, con todas sus etapas aprobadas, incluyendo la puerta de calidad de SonarQube. Las sesenta y ocho pruebas unitarias y las diez ejecuciones funcionales pasaron sin errores, lo que confirma que la lógica de negocio del sistema se comporta correctamente frente a los escenarios probados. Esto representa un resultado satisfactorio y una base sólida sobre la cual seguir construyendo.

En segundo lugar, la cobertura de código se distribuyó de manera desigual entre los paquetes del proyecto. La capa de servicios alcanzó valores excelentes, mientras que los controladores quedaron sin cobertura unitaria. Esta observación llevó a una lección importante: la cobertura por sí sola no es un indicador completo de calidad. Lo que importa no es alcanzar un número alto, sino asegurar que las partes críticas del sistema están bien probadas, y que las pruebas son significativas y no triviales. Una mejora natural para futuras iteraciones sería incorporar pruebas de integración web con MockMvc para cubrir los controladores con pruebas más rápidas que las E2E.

En tercer lugar, las pruebas de rendimiento expusieron dos cuellos de botella concretos: el problema N+1 en las consultas de tareas y la saturación del pool de conexiones bajo cargas elevadas. Estos hallazgos son particularmente valiosos porque muestran que las pruebas no son solo un mecanismo de verificación, sino también una herramienta de diagnóstico que permite tomar decisiones técnicas basadas en evidencia. Antes de pensar en escalar la aplicación horizontalmente, conviene atacar primero estos puntos específicos, lo cual probablemente resulte en una mejora desproporcionadamente alta del rendimiento.

En cuarto lugar, el análisis estático con SonarQube reveló una calidad de código buena pero perfectible. La ausencia total de duplicaciones, bugs y vulnerabilidades es un resultado positivo que refleja una arquitectura limpia y un uso consistente de buenas prácticas. Sin embargo, la presencia de veinte code smells, aunque la mayoría menores, indica que aún hay espacio para refinar detalles. La deuda técnica de dos horas y diecisiete minutos es manejable y debería resolverse antes de seguir agregando funcionalidades.

En quinto lugar, la integración de Docker y Jenkins demostró ser una elección acertada. Aunque la curva inicial de configuración fue empinada, especialmente para resolver la versión de Java y la comunicación entre contenedores, el beneficio a mediano plazo es enorme. Una vez configurado, el pipeline se ejecuta sin intervención manual y produce reportes consistentes en cada corrida, lo que ahorra tiempo y reduce errores humanos. Esta experiencia ilustra por qué la integración continua y la entrega continua se han convertido en estándares de facto en la industria.

Finalmente, una lección transversal a todo el proyecto es que la calidad de software no es un único concepto, sino la combinación armónica de muchos pequeños esfuerzos en distintos niveles. Las pruebas unitarias atrapan errores en piezas individuales del código, las pruebas funcionales validan los flujos de usuario, las pruebas de rendimiento miden el comportamiento bajo presión, y el análisis estático examina la estructura del código sin ejecutarlo. Ninguna de estas técnicas por sí sola garantiza calidad, pero combinadas dentro de un pipeline automatizado ofrecen un nivel de confianza difícil de alcanzar con pruebas manuales.

Como mejoras futuras se identifican cinco líneas de trabajo concretas: cubrir los controladores con pruebas de integración web, refactorizar el repositorio de tareas para eliminar el problema N+1, ajustar la configuración del pool de HikariCP, resolver los dos code smells críticos identificados por SonarQube, y revisar manualmente el security hotspot pendiente. Cada una de estas acciones es de bajo esfuerzo relativo y de alto impacto, lo que las convierte en candidatas ideales para una siguiente iteración del proyecto.

Referencias

- Apache Software Foundation. (2024). *Apache JMeter user manual*.
<https://jmeter.apache.org/usermanual/>
- Docker Inc. (2024). *Docker documentation*. <https://docs.docker.com/>
- Fowler, M. (2018). *Refactoring: Improving the design of existing code* (2nd ed.).
 Addison-Wesley.
- International Organization for Standardization. (2011). *ISO/IEC 25010:2011. Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*. ISO.
- International Organization for Standardization. (2022). *ISO/IEC/IEEE 29119-1:2022. Software and systems engineering — Software testing — Part 1: General concepts*. ISO.
- Jenkins Project. (2024). *Jenkins user documentation*. <https://www.jenkins.io/doc/>
- JUnit Team. (2024). *JUnit 5 user guide*. <https://junit.org/junit5/docs/current/user-guide/>
- Meszaros, G. (2007). *xUnit test patterns: Refactoring test code*. Addison-Wesley.
- Pressman, R. S., y Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9th ed.).
 McGraw-Hill Education.
- Selenium Project. (2024). *Selenium WebDriver documentation*.
<https://www.selenium.dev/documentation/>
- SonarSource. (2024). *SonarQube documentation*. <https://docs.sonarsource.com/sonarqube/>
- Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.
- VMware. (2024). *Spring Boot reference documentation*.
<https://docs.spring.io/spring-boot/docs/current/reference/html/>